BACKEND ENGINEERING • DAY 1

From Python Scripts to Your First API

Practical + Research-Oriented Learning

Learn → Understand → Set Up → Explore → Build → Explain

Your starting point: You are familiar with the Python concepts in your study PDF—functions, collections, modules/packages, files, JSON, exceptions, classes, inheritance and related fundamentals. You have no previous experience with web frameworks or databases. These notes start from there.

Today's goal

Understand what a backend is, how a client communicates with a server, what HTTP and an API mean, why a framework is useful, and how to prepare a clean environment to begin learning FastAPI.

Suggested time: 2.5–3.5 hours **Level:** Python fundamentals → first backend concepts

Learning rule: don't rush to copy code. Explain what each part is responsible for, what goes in, what comes out, and what can fail.

Table of Contents

Section	What you will learn
1. Your learning position	What you know already and what changes in backend work
2. What is a backend?	Client, server, business logic and data storage
3. HTTP and APIs	Requests, responses, methods, status codes and JSON
4. Why a framework?	What FastAPI provides—and what remains your responsibility
5. Set up the environment	Project folder, virtual environment, installation and server
6. Read the first example	Understand FastAPI's app, decorator, route and response
7. Practical lab	Run the official example and inspect the API
8. Architect thinking	Beginner mistakes, design boundaries and failure cases
9. Knowledge check	Questions, assignment, checklist and official sources

1. Your learning position

Your Python PDF taught you the language building blocks. Those same concepts still matter in backend development. A framework does not replace Python; it gives your Python program a way to receive requests from other programs and send responses back.

You already recognize	How it appears in a backend
Functions	Handle a request or perform a business operation.
Dictionaries and lists	Represent structured data and collections.
JSON	A common format for request and response bodies.
Modules and packages	Separate routes, business rules, validation and other responsibilities.
Exceptions	Represent and handle expected failures.
Classes and objects	Represent services, configuration, data models or framework objects.
Files and persistence	A first way to save data; later, a database handles shared structured data.

Knowing Python syntax is a starting point. Backend engineering adds networking, API contracts, databases, security, testing and operations.

2. What is a backend?

A backend is the part of an application that runs on a server. It receives requests, checks whether they are valid and allowed, applies business rules, reads or writes data, and returns a response.

Part	Simple meaning	Hospital ERP example
Client	The program making a request.	Patient app or browser.
Server	The program waiting for requests.	Your Python API application.
Business logic	Rules that decide what should happen.	A doctor cannot be booked twice for the same time.
Data store	Where records are kept.	Patient and appointment records in a database.
API	A defined way for programs to communicate.	An endpoint for viewing appointments.

3. HTTP: how a client talks to a backend

HTTP is a communication protocol used by clients and servers. Think of a request as a structured question sent to the server, and a response as the server's structured answer.

Request → server processes it → response

Request part	Meaning	Example idea
Method	The kind of action requested.	GET reads appointment information.
Path / URL	Which resource or operation is addressed.	/appointments/42 identifies one appointment.
Headers	Metadata about the request.	Content type or authentication information.
Body	Optional data sent with the request.	Details for a new appointment.

Common HTTP methods

Method	Typical use	Hospital example
GET	Read data; should not change server state.	Get a patient's appointment list.
POST	Create a resource or trigger an operation.	Request a new appointment.
PUT	Replace/update a resource according to the API contract.	Replace appointment details.
PATCH	Partially update a resource.	Change an appointment time.
DELETE	Remove a resource or request its deletion.	Cancel/delete a record, depending on design.

Status codes: the response's quick summary

Code	Meaning	Typical situation
200 OK	Request succeeded.	A record was returned.
201 Created	A resource was created.	Appointment created successfully.
204 No Content	Succeeded with no response body.	An update/deletion succeeded.
400 Bad Request	Request is invalid for the API.	Malformed or unacceptable input.
401 Unauthorized	Authentication is missing or invalid.	No valid login/token was supplied.
403 Forbidden	Identity is known but access is not allowed.	Patient tries an admin-only action.
404 Not Found	Resource or route was not found.	Appointment ID does not exist.
422 Unprocessable Content	Request data failed validation in many APIs.	Required field/type is invalid.
500 Internal Server Error	Unexpected server-side failure.	A bug or unhandled failure.

Do not memorize status codes as magic numbers. Learn what the client should understand and whether the operation actually succeeded.

4. JSON versus an API

JSON is a format for representing data. An API is the agreed interface through which a client asks for an operation. An API may use JSON, but JSON alone is not an API. In a web API, a client may send JSON to the server and receive JSON back.

Concept	Analogy
JSON	The format of a filled-in form.
HTTP request	The envelope carrying the form and requested action.
API endpoint	The service desk that accepts a specific type of request.
HTTP response	The reply stating the result and returning information.

5. Why use a framework?

You can build network servers using lower-level tools, but a web framework provides reusable infrastructure so you can focus on application behavior. FastAPI is a Python framework for building APIs. Its official First Steps guide introduces an application object, path-operation decorators, request handling and JSON-compatible responses.

Concern	What a framework helps provide
Incoming requests and routing	Map a URL path and HTTP method to a Python function.
Request parsing and validation	Declare expected data and validate incoming values.
Response behavior	Convert supported return values into HTTP responses.
API documentation	Generate interactive docs and an OpenAPI schema.
Reusable request-time setup	Provide mechanisms for shared resources and checks.

What FastAPI does not automatically solve

- It does not decide your business rules for you.
- It is not itself a relational database.
- It does not make every endpoint secure just because it is installed.
- It does not guarantee good architecture, correct transactions or high performance.
- It does not replace tests, logging, deployment knowledge or careful error handling.

Architect mindset: a framework is a tool, not the architecture. You still choose where business rules live, how data is stored, how permissions work and how failures are handled.

6. Read the first FastAPI example conceptually

The official tutorial's first example is intentionally small. Study the purpose of each part rather than trying to memorize the punctuation.

Piece	Meaning
Import the FastAPI class	Bring the framework's application class into your Python module. This is a normal Python import.
Create an app instance	Instantiate the class. The application object is the central object used to register API operations.
Use a decorator such as @app.get("/")	Register the function below it to handle GET requests for the root path. This uses Python decorator syntax you have already encountered.
Define the path-operation function	Write the Python function FastAPI calls when a matching request arrives.
Return a dictionary	FastAPI converts supported return values, including dictionaries, into a response—commonly JSON.
Run a development server	Start a local process that listens for HTTP requests while you learn and test.

Connect this to concepts you already know

- The app is an object created from a class.
- The endpoint handler is a Python function.
- The decorator registers framework behavior for that function.
- The returned dictionary is a familiar Python data structure.
- The JSON response is the transport representation of that data.
- The URL path and HTTP method select which function handles a request.

Follow the official First Steps page for the actual small example. These notes focus on understanding the concepts before expanding the code.

7. Set up your first framework environment

Use a separate project folder for this experiment. Do not install learning dependencies globally or mix the experiment into your existing Hospital ERP yet. Python's official documentation explains that virtual environments isolate packages for different projects.

Step 1 — Create a project folder

In VS Code, create a folder such as **backend-day-01**, open that folder, and open the integrated terminal. Keep the first exercise separate from your existing projects.

Step 2 — Create a virtual environment (Windows PowerShell)

```
python -m venv .venv
```

```
.venv\Scripts\Activate.ps1
```

The first command creates an isolated environment named .venv. The second activates it in PowerShell. If activation is blocked by your machine's execution policy, do not change system security settings blindly; use the interpreter inside .venv directly or follow supported VS Code environment guidance.

Step 3 — Install FastAPI

The current official FastAPI tutorial recommends using the uv project tool. For a simple first experiment, you can use pip inside the active virtual environment. Choose one approach and avoid mixing methods without understanding which environment they target.

```
python -m pip install "fastapi[standard]"
```

The quoted package extra installs FastAPI along with standard optional components used in the getting-started workflow. The quotes matter in shells where square brackets have special meaning.

Step 4 — Follow the official First Steps tutorial

- Open the official First Steps link in the reference section.
- Create the tutorial's small main.py example exactly as the page explains.
- Save the file inside your new project folder.
- Start the development server using the command documented by the current tutorial.
- Keep the terminal running while you test the endpoint in a browser.

Step 5 — Inspect the running application

Local URL	What to inspect
http://127.0.0.1:8000/	Response from the root endpoint.
http://127.0.0.1:8000/docs	Interactive API documentation (Swagger UI).
http://127.0.0.1:8000/redoc	Alternative API documentation (ReDoc).
http://127.0.0.1:8000/openapi.json	Generated OpenAPI description represented as JSON.

These addresses work only while the development server is running on the default local port. They are local to your machine, not public internet addresses.

8. Practical lab — learn by observing

Do not stop after seeing the first response. Your job today is to build a mental model of what the framework is doing.

Lab A — Observe a request

- Start the example and open the root URL in your browser.
- Write down the exact response and identify the JSON object.
- Open /docs and find the root operation.
- Identify its HTTP method, path and expected response.
- Open /openapi.json and notice that it describes the API rather than implementing it.

Lab B — Predict before testing

Experiment	Question to answer
Visit a path you did not register.	What response status appears, and why?
Use the interactive docs to call the root endpoint.	Does it return the same content as the browser?
Stop the development server and refresh.	Why can the browser no longer reach the endpoint?
Change the returned message in your own example.	Which part changed: route selection, business logic or response data?
Read the terminal output while making a request.	What does the server log show?

Lab C — Draw the request lifecycle

Browser / API client → HTTP request → development server → matching route → Python function → returned data → HTTP response → client

Lab D — Connect it to Hospital ERP

Do not build the whole hospital system today. Design a future read-only endpoint for listing doctors or appointments. State the path, method, response shape, and what should happen when no matching record exists.

Design question	Your answer
What resource will the endpoint represent?	_____
What HTTP method should read it?	_____
What data should a successful response contain?	_____
What should happen for a missing ID?	_____
Who should be allowed to access it?	_____

9. Beginner mistakes to avoid

Mistake	Better engineering habit
Thinking FastAPI replaces Python.	It is a framework used with Python.
Thinking a URL is the full API contract.	Consider method, path, input, output, status and permissions.
Thinking JSON is a database.	JSON is a format; a JSON file is not a full multi-user database system.
Putting every operation inside one route function.	Keep request handling thin and business rules separately testable as the project grows.
Installing packages into an unknown environment.	Activate the intended environment and verify which interpreter runs.
Treating a browser response as proof of production readiness.	You still need validation, authorization, tests, persistence, logging and deployment.
Starting with async, databases, authentication and deployment at once.	Learn one layer at a time, then integrate deliberately.

10. Today's knowledge check

Answer without opening the notes. If you cannot explain an answer, revisit the relevant section and test the concept.

Q1. What is the difference between a client and a server?

_____

Q2. What is an API, and how is it different from JSON?

_____

Q3. What are the method and path in an HTTP request?

_____

Q4. When would you typically use GET versus POST?

_____

Q5. What does a 404 response communicate?

_____

Q6. What is the role of the FastAPI app instance?

_____

Q7. What does a path-operation decorator register?

_____

Q8. Why can a returned dictionary become a JSON response?

_____

Q9. Why should each project use a virtual environment?

_____

Q10. What does /docs provide, and what does OpenAPI describe?

_____

Q11. What problems does FastAPI help with—and what problems remain your responsibility?

_____

11. Today's assignment

- **Required:** complete the official FastAPI First Steps tutorial and run the small example locally.
- **Required:** visit the root URL, /docs and /redoc; describe what each shows.
- **Required:** draw the request lifecycle and label client, HTTP request, route, function, response and JSON.

- **Required:** write short answers to the 11 knowledge-check questions.
- **Optional:** design three future Hospital ERP endpoints: list doctors, view one doctor, and list appointments. Do not add a database today.

Completion checklist

- ■ I can explain what a backend does.
- ■ I understand the basic HTTP request/response cycle.
- ■ I can distinguish a method, path, status code and JSON body.
- ■ I understand the role of a framework and FastAPI.
- ■ I created and activated a project-specific virtual environment.
- ■ I ran the official first FastAPI example.
- ■ I inspected the API docs and OpenAPI schema.
- ■ I can explain the example without memorizing it.

12. Official documentation — primary references

These notes introduce concepts, but the official documentation should remain your source of truth. Follow the current documentation because commands and recommended tooling can evolve.

FastAPI — First Steps
https://fastapi.tiangolo.com/tutorial/first-steps/
First application, routes, server startup, interactive docs and OpenAPI.

FastAPI — Tutorial / User Guide
https://fastapi.tiangolo.com/tutorial/
Recommended step-by-step learning path after today's lesson.

Python — Virtual Environments and Packages
https://docs.python.org/3/tutorial/venv.html
Why environments exist and how Python packages are installed.

Python — venv reference
https://docs.python.org/3/library/venv.html
Details about creating isolated Python environments.

MDN — Overview of HTTP
https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview
Beginner-friendly explanation of HTTP and client/server communication.

OpenAPI Specification
https://spec.openapis.org/oas/latest.html
Standard used to describe HTTP APIs in a machine-readable form.

Where the next lessons go

Lesson	Focus	Outcome
Day 2	HTTP methods, paths, query/path parameters and request/response models	Understand the API contract.
Day 3	Pydantic validation and useful error responses	Reject invalid inputs deliberately.
Day 4	Organizing a FastAPI project into modules and routers	Separate responsibilities.
Days 5–6	SQL and relational database fundamentals	Understand tables, keys and relationships before ORM abstractions.
Later	Database integration, tests, authentication, migrations and deployment	Grow the API into a reliable service.

Today's success is not “I know FastAPI.” It is “I understand the client/server model, ran one endpoint, inspected its behavior, and can explain how my existing Python knowledge fits into it.”

Prepared for a Python learner beginning backend development. This guide deliberately starts before databases and assumes no previous framework experience.